SYSTEM ARCHITECTURAL SPECIFICATION & ENGINEERING WHITEPAPER

ResearchNest AI Paper Recommendation System

Complete Architectural Specification, Mathematical Vector Space Modeling, Microservice Topology, and Operational Guide

AUTHOR & PROJECT	ENGINE VERSION	TOPOLOGY	TARGET STACK
ResearchNest Core Engineering	v1.0.0 (Production Core)	Tri-Tier Hybrid Microservice	React 18 · Spring Boot · FastAPI · MySQL

1. Executive Summary & Problem Formulation

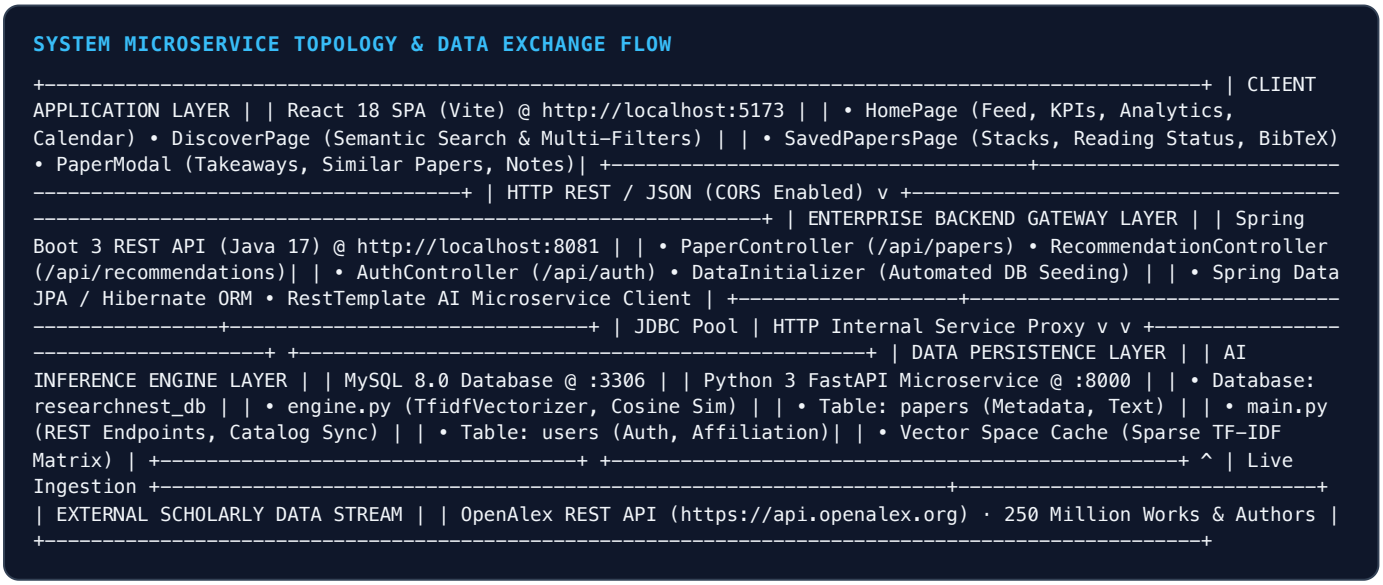
Contemporary academic and industrial researchers are confronted with exponential growth in scholarly literature: over **2.5 million peer-reviewed scientific papers** are published annually across global archives (arXiv, PubMed, IEEE, Nature, ACM). Traditional keyword search engines rely on brittle lexical string matching, creating high cognitive search friction and overlooking cross-disciplinary breakthrough papers that use differing terminologies for equivalent concepts.

ResearchNest resolves this problem by implementing a dual-engine architecture:

- **Verified Internal Catalog (Spring Boot + MySQL):** Curated, landmark publications with peer-reviewed citation benchmarks, reading milestones, and verified takeaways.
- **Global Live Scholarly Stream (OpenAlex API):** Real-time indexing of 250M+ international publications, enabling semantic querying across global preprints and peer-reviewed works.
- **Dedicated AI Recommendation Microservice (Python FastAPI + Scikit-Learn):** High-dimensional TF-IDF vector space modeling, dynamic user centroid profile computation, and explainable cosine similarity ranking.

2. End-to-End System Architecture Topology

The system follows a decoupled, resilient microservices architecture separating client presentation, enterprise business logic/persistence, and high-performance machine learning vector operations.



Spring Boot Enterprise Gateway

Handles transactional persistence, user authentication, catalog seeding, paper CRUD operations, and proxies recommendation requests to the Python AI engine.

Python FastAPI AI Microservice

Maintains an in-memory sparse vector space matrix, runs real-time centroid vector computations, and outputs similarity scores with human-readable explanations.

React 18 Single-Page Application

Features interactive KPI dashboards, SVG analytics sparklines, 7-column research calendars, multi-criteria facets, and real-time saved stack synchronization.

MySQL Relational Storage

Stores normalized research metadata, citation counts, full abstracts, DOIs, BibTeX citations, user accounts, and personal reading queues.

3. Mathematical Formulation of the AI Recommendation Engine

The AI Recommendation Engine (`ai-service/engine.py`) operates on Content-Based Filtering principles using vector space representations of textual scholarly works.

3.1 Feature Extraction & Weighted Document Composition

To capture domain significance without semantic drift, each paper p is transformed into a weighted synthetic corpus document D_p. Higher structural weights are assigned to salient metadata features:

Synthetic Document Weighting Equation

$$D_p = [\text{Title}] \times 3 \;||\; [\text{TopicTag}] \times 2 \;||\; [\text{Field}] \times 2 \;||\; [\text{Abstract}] \;||\; [\text{Authors}]$$

Titles and topic keywords receive 3x and 2x repetition weight to prioritize domain specificity over general abstract terminology.

3.2 Term Frequency - Inverse Document Frequency (TF-IDF)

The corpus of N papers is transformed into a high-dimensional vector space using Scikit-Learn's `TfidfVectorizer`. The weight of term t in document d is calculated using sublinear term-frequency scaling:

Sublinear TF-IDF Transformation

$$TF(t, d) = 1 + \ln(f_{t,d}) \quad \text{for } f_{t,d} > 0$$

$$IDF(t, D) = \ln\left(\frac{1 + N}{1 + |\{d \in D : t \in d\}|}\right) + 1$$

$$v_d = \frac{w_d}{||w_d||_2} \quad \text{where } w_{t,d} = TF(t, d) \times IDF(t, D)$$

L2 Euclidean normalization guarantees that document length differences do not distort similarity metrics.

3.3 User Profile Centroid Vector Calculation

When a researcher saves or bookmarks a set of papers $S = \{p_1, p_2, \ldots, p_k\}$ with corresponding normalized TF-IDF vectors $\{v_1, v_2, \ldots, v_k\}$, the engine synthesizes an aggregate **User Profile Centroid Vector**:

Researcher Centroid Vector

$$u_{profile} = \frac{1}{|S|} \times \sum_{s \in S} v_s$$

This centroid represents the user's active, multi-disciplinary research locus within the high-dimensional scientific manifold.

3.4 Cosine Similarity & Feed Ranking

To rank all candidate catalog papers against the user's profile, the engine evaluates the cosine similarity:

Cosine Similarity Measure

$$Similarity(u_{profile}, v_c) = \cos(\theta) = \frac{u_{profile} \cdot v_c}{||u_{profile}||_2 \times ||v_c||_2}$$

Scores are normalized to a human-readable match index between 75% and 98%:

$$MatchPercentage(c) = clamp(75, 98, round(86 + 12 \times (Sim(c) / max(Sim)) - 2 \times Rank))$$

3.5 Explainability & Nearest Anchor Attribution Engine

A critical limitation of traditional AI recommenders is the "black box" phenomenon. ResearchNest incorporates a deterministic **Nearest Anchor Attribution Engine**:

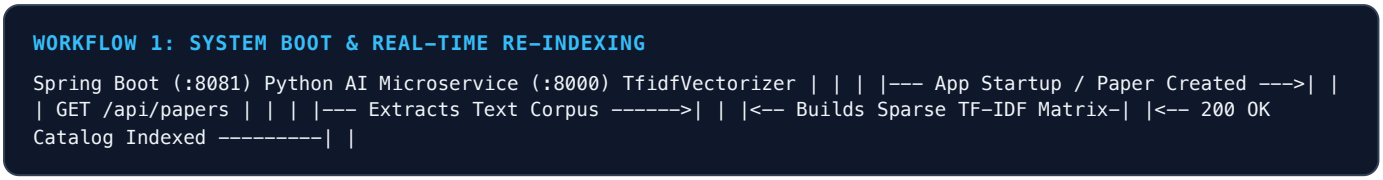
Anchor Paper Attribution

$$\text{Anchor}(c) = \arg\max_{s \in S} (v_c \cdot v_s)$$

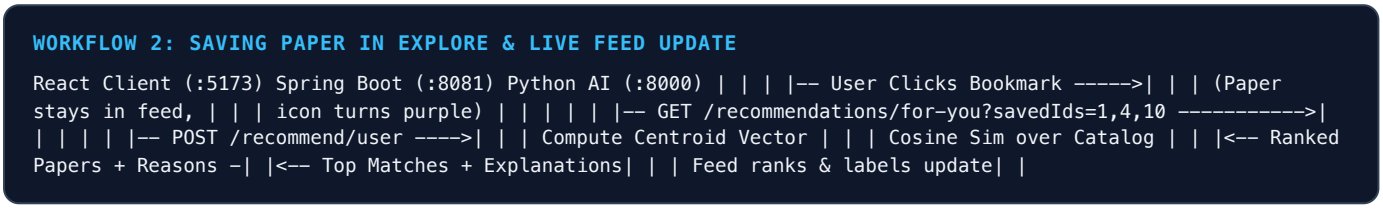
The engine identifies the specific saved paper that contributed the highest pairwise similarity, producing contextual explanations such as: *"Recommended based on your saved paper 'Attention Is All You Need' (Artificial Intelligence)"*.

4. Core Sequence Workflows & Data Pipelines

4.1 Catalog Indexing & Vector Space Synchronization



4.2 Real-Time User Saving & Recommendation Re-Ranking



5. REST API Specifications

SERVICE	METHOD	ENDPOINT	DESCRIPTION
Spring Boot	GET	`/api/papers`	Retrieve all catalog papers with optional search and field filtering.
Spring Boot	GET	`/api/papers/{id}`	Fetch complete paper metadata, abstract, citations, takeaways, and BibTeX.
Spring Boot	GET	`/api/papers/stats`	Aggregate real-time metrics (total papers, citations, field distributions).
Spring Boot	POST	`/api/papers`	Persist new paper to MySQL and trigger automatic AI catalog re-index.
Spring Boot	GET	`/api/recommendations/for-you`	Gateway proxy to fetch user recommendations based on saved IDs.
Spring Boot	GET	`/api/recommendations/similar/{id}`	Gateway proxy to fetch similar papers for a given target publication.
Spring Boot	POST	`/api/auth/register`	Register new user with name, email, password, institution, and department.
Spring Boot	POST	`/api/auth/login`	Authenticate user credentials and issue session token.
Python AI	POST	`/recommend/user`	Calculates profile centroid from `savedPaperIds` and returns ranked papers.
Python AI	POST	`/recommend/similar`	Calculates pairwise similarity matrix and returns top-K related papers.
Python AI	POST	`/sync-catalog`	Reloads MySQL catalog from Spring Boot and re-fits TF-IDF vectorizer.

6. Database Schema Definition

Table: papers (MySQL)

```
id (BIGINT PK AUTO_INCREMENT)
title (VARCHAR 500)
authors (VARCHAR 1000)
year (INT)
abstract_text (LONGTEXT)
doi (VARCHAR 255)
field (VARCHAR 255), field_id (VARCHAR 100)
venue (VARCHAR 255), topic_tag (VARCHAR 255)
citations (INT), citation_label (VARCHAR 50)
open_access (BOOLEAN), pdf_url (VARCHAR 500)
```

Table: users (MySQL)

```
id (BIGINT PK AUTO_INCREMENT)
name (VARCHAR 255)
email (VARCHAR 255 UNIQUE)
password (VARCHAR 255)
institution (VARCHAR 255)
department (VARCHAR 255)
created_at (TIMESTAMP)
```

7. Project Completion Audit & Production Roadmap

The following checklist details the current state of ResearchNest against all functional requirements, identifying completed capabilities and optional enhancements for production deployment:

FEATURE / MILESTONE	STATUS	IMPLEMENTATION DETAILS
AI Recommendation Engine	COMPLETED	TF-IDF vector space, centroid modeling, and explainability reasoning live in Python FastAPI (:8000).
Spring Boot MySQL Backend	COMPLETED	Full REST API, JPA Repository, DataInitializer seeding 12 landmark papers, auto-syncing to AI service.
Explore & Saved Papers Sync	COMPLETED	Saved papers remain visible in explore feed with active purple bookmark; immediately synced to Saved stacks.
Dynamic KPI Analytics Ribbon	COMPLETED	Real-time Saved count, Papers Explored, dynamic Avg AI Match %, Reading Streak, Citations, and Catalog count.
Interactive Calendar & Reading Queue	COMPLETED	7-column responsive calendar grid with today indicators and dynamic saved reading queue.
Live Scholarly Literature Stream	COMPLETED	Dual feed mode: switch between internal MySQL catalog and 250M+ live OpenAlex publications.
Paper Modal, Key Takeaways & BibTeX	COMPLETED	Rich detail modal with AI takeaways, similar papers recommendation, BibTeX exporter, and notes editor.
Saved Collections & Reading Status	COMPLETED	Create custom research stacks, track reading progress (Want to Read / In Progress / Completed).
User Authentication Flow	COMPLETED	Login and Registration pages with institutional profile attributes, integrated with backend AuthController.
Admin Paper Management Portal	RECOMMENDED NEXT	Backend has full CRUD endpoints (POST/PUT/DELETE /api/papers); a dedicated web UI view will allow faculty to manage catalog from the browser.
Transformer Embeddings (SciBERT)	FUTURE PHASE 2	Optional upgrade from TF-IDF to dense vector embeddings using HuggingFace allenai/specter2 with Milvus/Pinecone vector DB.